

HET CLAUDE HANDBOEK

Niveau Één

Van taken uitvoeren naar echte dingen bouwen.

Met schema's, concrete voorbeelden en stap-voor-stap-procedures.

Prompting · Projects · Skills · Connectors · Data & GDPR · Source & Git · Code · Agents

Het vervolg op Niveau Nul — voor wanneer de basis makkelijk aanvoelt.

Door Hans Tandt

Ardennen, Wallonië, België

Versie 2.1 — Augustus 2026

Hoe je dit boek gebruikt

Dit is Niveau Één. Het veronderstelt dat je de dingen van Niveau Nul gedaan hebt: Claude instellen, een taak draaien in Cowork, je bestanden terugvinden, een sessie beheren. Zo niet, lees eerst Niveau Nul — dit boek bouwt daarop voort.

Niveau Één is de overgang van losse taken naar het bouwen van dingen die blijven: beter prompten, herbruikbare skills, verbonden tools, beheerste data, gedeelde broncode, een stap in Code, en je eerste kleine agent.

Drie routes — lees wat je nodig hebt

Nieuw in dit alles: lees in volgorde.

Je hebt de basis en wilt kwaliteit: hoofdstukken 1–5 (prompting, projects, skills, connectors, data & GDPR).

Je hebt teamwerk nodig: ga naar hoofdstukken 7–9 (source, Git, team) — ze staan op zichzelf.

Dit boek is een momentopname — Claude verandert snel

Alles hier weerspiegelt Claude in augustus 2026: de modelfamilies Opus, Sonnet, Haiku en Fable, de Cowork desktop-app, en het verenigde home dat Chat, Cowork en je Projects in één zijbalk deelt. Anthropic brengt updates zeer snel uit — vaak in dagen, niet maanden — dus een scherm, een menunaam of een modelnummer kan al verschoven zijn. Als wat je ziet niet met de pagina overeenkomt, vertrouw dan de app en claude.com. De ideeën en gewoonten hier houden stand, ook als de knoppen verhuizen.

Wat is nieuw in versie 2.1

Toegevoegd: « Record a skill » in Cowork (hoofdstuk 3) en Claude Design (hoofdstuk 10). Bijgewerkt: waar Projects nu leven in het nieuwe enkele home, en een herinnering dat modellen en schermen snel veranderen.

Inhoudsopgave

Hoe je dit boek gebruikt.....	2
Inhoudsopgave.....	3
Niveau Nul op één pagina.....	5
1. Prompten dat echt werkt.....	6
1.1 Context in actie — voor / na.....	6
1.2 Het anti-stijlbestand.....	6
1.3 Tonen, dan corrigeren.....	6
1.4 Het antwoord verifiëren — wat, en hoe.....	6
2. Projects in de diepte.....	7
2.1 Eén project per resultaat.....	7
2.2 Wat de instructies bevatten.....	7
2.3 Procedure — een nuttig project maken in 5 minuten.....	7
3. Je eerste Skill bouwen.....	8
3.1 Wat een skill echt is.....	8
3.2 Procedure — een skill maken en testen.....	8
4. Connectors — Claude je andere tools laten bereiken.....	9
4.1 Verbind alleen wat de taak nodig heeft.....	9
4.2 De drie soorten connector.....	9
4.3 Een veilig eerste gebruik.....	9
5. Data, veiligheid en GDPR voor je verbindt.....	10
5.1 De data-checklist voor je verbindt.....	10
5.2 GDPR in gewone taal.....	10
5.3 Traceren en de controle houden.....	10
6. Meerdere projecten parallel voeren.....	11
6.1 Wat tegelijk kan draaien.....	11
6.2 De twee echte kosten.....	11
7. Waar de broncode van je project leeft.....	12
7.1 Een eenvoudige regel.....	12
7.2 Houd een nette structuur.....	12
8. Een zachte kennismaking met Git.....	13
8.1 Wat Git echt doet.....	13
8.2 De dagelijkse lus.....	13
8.3 Werksessie — je eerste repo in 10 minuten.....	13
9. In team werken aan hetzelfde project.....	14
9.1 Documenten of code.....	14
9.2 De Git-lus in team.....	14
10. Een zachte eerste stap in Code.....	15
10.1 Beschrijven, niet typen.....	15
10.2 CLAUDE.md — het brein van het project.....	15
10.3 De permissies — drie niveaus, niet twee.....	15
10.4 Eerst plannen, dan bouwen.....	15
10.5 Vibe coding — en zijn grenzen.....	15
10.6 Claude Design — interfaces zonder code.....	15
11. Je eerste kleine agent.....	16
11.1 Wat een agent maakt.....	16
11.2 Een volledig geval — een ochtendbriefing.....	16
11.3 Laat hem langzaam groeien.....	16
12. Tools en apps om te installeren.....	17
13. Onderhoudbaar blijven.....	18
Woordenlijst van nieuwe termen.....	19
Wat volgt.....	20
De API — de stap voorbij dit boek.....	20

Niveau Nul op één pagina

Voor je bouwt, de vijf ideeën waarop alles rust:

1. **Kies het juiste gezicht.** Chat om te denken, Cowork om te produceren, Code om te bouwen, Projects om te onthouden, Connectors om andere apps te bereiken.
2. **Context verslaat de truc.** Wat Claude ziet vóór het antwoordt telt meer dan de formulering.
3. **Laat Claude eerst vragen, verifieer dan.** « Stel me eerst vragen »; en vertrouw nooit een zelfverzekerd antwoord zonder het te checken.
4. **Een bestand is pas echt in jouw map.** Zolang het er niet staat, leeft het in de sandbox of scratchpad en kan het verdwijnen.
5. **Houd sessies kort.** Eén onderwerp per sessie; sessienotities en geheugen zodat je jezelf nooit opnieuw hoeft uit te leggen.

1. Prompten dat echt werkt

Niveau Nul zei: context verslaat de truc. Niveau Één maakt dat concreet. Een goede vraag heeft vier delen, en geen enkel deel is een toverwoord.

Een goede opdracht heeft vier delen (geen toverwoord)



Geef de bestemming en de normen — geen lange lijst stappen.

Deel	Wat het betekent	Voorbeeld
Doel	Het gewenste resultaat, in één zin.	« Schrijf een offertebrief van één pagina. »
Context	De nodige feiten, bestanden en beperkingen.	« Gebruik het bijgevoegde sjabloon en onze gebruikelijke voorwaarden. »
Voorbeelden	Eén of twee stalen van goede (en slechte) output.	« Neem de toon van deze oude brief over; vermijd deze wending. »
Kwaliteitslat	Hoe je oordeelt; wat « af » betekent.	« Formeel, minder dan 250 woorden, geen jargon. »

1.1 Context in actie — voor / na

Dezelfde taak, twee prompts. **Voor:** « Schrijf een e-mail aan een leverancier over de late bestelling. » Je krijgt een generieke e-mail. **Na:** « Schrijf aan onze leverancier Dupont, bestelling 4471, 9 dagen te laat. Streng maar beleefd, vraag een nieuwe datum en een gedeeltelijke creditnota, minder dan 120 woorden, teken in mijn naam. » Je krijgt iets dat bijna klaar is. Het verschil is niet de truc — het is de context.

1.2 Het anti-stijlbestand

De snelste manier om output te stoppen die « naar AI ruikt » is opschrijven wat je nooit wilt. Een kort « anti-stijl »-bestand — verboden woorden, gehate openingen, afgewezen wendingen — doet meer dan welke positieve instructie ook. Bewaar het in ABOUT ME.

1.3 Tonen, dan corrigeren

Mik niet op de perfecte treffer in één keer. Vraag een klad, reageer, laat Claude bijstellen. Drie rondes « goed, maar korter en minder formeel » verslaan dertig minuten aan de perfecte prompt sleutelen.

1.4 Het antwoord verifiëren — wat, en hoe

Claude klinkt even zeker als het fout zit. Houd Web Search aan, en vraag het zichzelf te controleren: « *Verifieer je eigen antwoord voordat ik erop vertrouw; check de feiten en zeg waar je niet zeker van bent.* »

2. Projects in de diepte

Een Project is een map met gesprekken die instructies, bestanden en geheugen delen. Zo maak je er één die tijd bespaart in plaats van een rommelbak te worden. Sinds medio 2026 leven je projects in de linker zijbalk van het verenigde home, gedeeld tussen Chat en Cwork — pin de projecten die je gebruikt zodat je ze makkelijk terugvindt.

2.1 Eén project per resultaat

De gangbare fout is één reuzenproject voor « alles ». De oplossing: één project per terugkerend resultaat — een weekrapport, een soort voorstel, één specifiek hulpmiddel. Elk krijgt alleen zijn eigen bestanden en instructies geschreven voor dat ene werk.

2.2 Wat de instructies bevatten

Instructies zijn geen documentatie — het is het minimum dat Claude belet af te dwalen. Vijf tot acht regels:

- Wat dit resultaat is, en voor wie.
- De toon en de kwaliteitslat.
- Wat altijd te doen, en wat nooit.
- Waar het resultaat op te slaan, en in welk formaat.

Bestanden zijn niet gratis

Meer bestanden betekent geen betere antwoorden — wel meer ruis om te filteren. Stuur stalen van goed werk en je sjablonen, niet je hele archief. Als een antwoord afdwaalt, is het vaak te veel bestanden, niet te weinig.

2.3 Procedure — een nuttig project maken in 5 minuten

1. Noem het naar een resultaat (« Wekelijks ops-rapport »).
2. Voeg alleen zijn sjablonen en 1–2 goede eerdere voorbeelden toe.
3. Schrijf 5–8 regels instructies (wat / voor wie / toon / waar opslaan / wanneer vragen).
4. Test met een echte taak; laat het zichzelf bekritisieren en scherp de instructies aan.

3. Je eerste Skill bouwen

Een Skill is jouw standaardmanier om een werk te doen, opgeschreven zodat Claude het elke keer hetzelfde uitvoert. Als je ooit een checklist schreef, kun je er een bouwen — geen code nodig.

3.1 Wat een skill echt is

Een skill is een klein tekstbestand met de naam `SKILL.md`. Een korte omschrijving bovenaan, de volledige instructies eronder. Alleen de omschrijving blijft in het geheugen; de rest laadt wanneer een taak past. Daarom kun je er veel hebben zonder iets te vertragen.

3.2 Procedure — een skill maken en testen

1. **Doe de taak één keer met de hand** met Claude, en noteer de stappen en fouten.
2. **Oogst het:** « Maak van wat we net deden een herbruikbare skill. Schrijf de stappen helder op, voeg een korte sectie valkuilen toe, en houd de omschrijving op één regel. »
3. **Test op een vers voorbeeld** dat lichtjes afwijkt, en corrigeer wat de skill miste.
4. **Bewaar het** in de app, onder Settings, Capabilities. Een skill verbetert alleen als je zijn bestand bewerkt.

De snelste manier om er een te maken: neem jezelf op terwijl je het doet

Sinds medio 2026 heeft Cowork een « Record a skill »-optie in het +-menu van de desktop-app. In plaats van de stappen te schrijven, neem je je scherm op en praat je over de taak terwijl je hem één keer doet, en Claude maakt van de demonstratie de skill. Het is de makkelijkste weg voor een taak die eenvoudiger te tonen dan te beschrijven is — vooral browsertaken zonder connector. Neem geen wachtwoorden of privé-data op. Op Pro, Max en Team.

4. Connectors — Claude je andere tools laten bereiken

Een connector is een beveiligde brug tussen Claude en een andere app — mail, agenda, een drive, een projectopvolging. Met een actieve connector kan Claude data buiten je map lezen en erop handelen.

4.1 Verbind alleen wat de taak nodig heeft

De reflex is alles te verbinden. Weersta die. Elke connector voegt ruis toe en verbruikt context. Een ochtendbriefing heeft mail en agenda nodig — niet je hele gereedschapskist. Zet uit wat een stroom niet gebruikt.

4.2 De drie soorten connector

Connectors rusten op een gemeenschappelijke standaard, **MCP** (Model Context Protocol). Je voegt ze toe via de Connectors-instellingen (de « + » in een gesprek, of Settings, Connectors). Er zijn drie soorten, en het verschil telt voor vertrouwen:

- **De ingebouwde integraties van Anthropic** — eerste partij (Gmail, Drive, GitHub, Slack...).
- **De connectors uit de directory** — derde partij, maar onderzocht en gevalideerd door Anthropic vóór ze worden vermeld.
- **De aangepaste MCP-connectors die je zelf toevoegt** — niemand heeft ze onderzocht; dezelfde voorzichtigheid als bij elke nieuwe app.

4.3 Een veilig eerste gebruik

Begin alleen-lezen. Vraag om de ongelezen mail van vandaag samen te vatten, of de vergaderingen van de week te lijsten, voordat je Claude laat versturen of wijzigen. Bouw vertrouwen op onschuldige taken, en breid dan uit.

5. Data, veiligheid en GDPR voor je verbindt

Zodra Claude echte bedrijfsdata raakt — via een connector, een geplakt document of een map — wordt gegevensbescherming jouw verantwoordelijkheid. Dit hoofdstuk is praktische bewustwording, geen juridisch advies; bij twijfel, vraag je functionaris voor gegevensbescherming (DPO).

5.1 De data-checklist voor je verbindt

1. Welke data zal Claude zien, en identificeren sommige een persoon?
2. Heb ik een reden (een rechtsgrond) om ze zo te verwerken?
3. Is dit het strikte minimum, of deel ik te veel?
4. Kan ik anonimiseren of afschermen en toch het resultaat krijgen?

5.2 GDPR in gewone taal

De EU-GDPR omkadert persoonsgegevens — alles wat een levende persoon identificeert (namen, e-mails, personeels- of klantdossiers, een ongevals dossier). Enkele principes dekken het meeste:

- **Heb een reden.** Verwerk persoonsgegevens alleen voor een duidelijk, legitiem doel.
- **Het minimum.** Breng niet meer persoonlijke details in dan de taak vereist.
- **Blijf bij het doel.** Hergebruik de data niet voor iets onverwachts.
- **Bescherm en wis.** Houd ze veilig, en niet langer dan nodig.

Waar gaat de data heen? (en kan ik ze lokaal houden?)

Chat, Cowork en Code sturen je inhoud allemaal naar Anthropic's servers. Claude draait niet op je computer, en geen instelling houdt je data lokaal. Je KUNT het gebruik van je gesprekken om de modellen te trainen uitzetten (Settings, Privacy), wat ook de bewaring verkort. Team-, Enterprise- en API-accounts trainen standaard niet op je data. Voor echt lokale, offline AI heb je een ander, zelf-gehost model nodig — niet Claude.

5.3 Traceren en de controle houden

Houd voor alles wat gevoelig is een licht spoor bij van wat je gaf en waarom, geef de voorkeur aan de verkeerslichtregel van Niveau Nul (groen / oranje / rood), en zorg dat een mens controleert vóór elke verzending of actie. Als een connector kan handelen (mail sturen, dossiers wijzigen), begin alleen-lezen.

Geen juridisch advies

Dit hoofdstuk is algemene bewustwording om de voor de hand liggende fouten te vermijden. GDPR-verplichtingen hangen af van je rol en je organisatie — bevestig de details met je DPO voordat je persoonsgegevens op grote schaal verwerkt.

6. Meerdere projecten parallel voeren

Je bent niet beperkt tot één ding tegelijk. Je kunt een document schrijven in Cowork terwijl Claude een app repareert in Code, en meerdere Cowork-taken in beweging houden. De tools draaien onafhankelijk — de grens ben jij, en je quota.

6.1 Wat tegelijk kan draaien

- **Cowork en Code samen.** Twee vensters, twee werken, zonder interferentie.
- **Meerdere sessies.** Verschillende gesprekken of projects, elk met eigen context, naast elkaar.
- **Achtergrond- en geplande taken.** Werk dat vordert zonder toezicht, en zelfs Dispatch-taken vanaf de telefoon.

6.2 De twee echte kosten

1. **Je aandacht.** De machines mengen niet, maar jij wel. Elke minuut van een document naar een bug springen is vermoeiend. Groepeer per context: een uur schrijven, dan een uur op de app.
2. **Je quota.** Twee grote werken putten uit dezelfde limieten (5 uur en week). Ze samen voeren verbruikt je quota sneller.

7. Waar de broncode van je project leeft

Zodra een project telt — of meerdere mensen eraan werken — houdt de plaats waar de bestanden leven op een detail te zijn.

Waar de broncode van je project hoort te leven

Een lokale map	OneDrive / SharePoint	Een Git-repository
Solo werk, proeven. Maak een back-up.	Solo of licht delen. Auto-back-up. Niet om code samen te voegen.	Code, volledige historie, echt teamwerk. Kleine leercurve.

Alleen op documenten → lokaal + OneDrive. Code, of een team → Git.

7.1 Een eenvoudige regel

- **Alleen aan documenten?** Een lokale map met OneDrive-back-up volstaat.
- **Alleen aan code?** Gebruik toch Git — een volledige geschiedenis en een « ongedaan maken » voor het hele project.
- **Met meerderen aan dezelfde bestanden?** Git voor code; een SharePoint- of Drive-bibliotheek voor documenten.

7.2 Houd een nette structuur

Wat je ook kiest, richt Cwork of Code op een nette map met een vanzelfsprekende vorm: bronbestanden, een plek voor de output, een notitie- of README-bestand, en niets vreemds. Een nette map maakt Claude sneller en je collega's geruster.

8. Een zachte kennismaking met Git

Git geeft een project een geheugen en een vangnet. Het is het nuttigste om te leren bij de overgang van solo-experimenten naar echt gedeeld werk — en veel minder eng dan het lijkt.

8.1 Wat Git echt doet

Git neemt momentopnamen van je project wanneer je het vraagt. Elke opname (een « commit ») is een gelabeld punt om naar terug te keren. Je kunt dus alles veranderen zonder vrees: breekt het, dan ga je terug naar de laatste goede opname. Niets gaat ooit echt verloren.

Woord	In gewone taal
Repository (repo)	Je projectmap, plus heel zijn geschiedenis van opnamen.
Commit	Een opgeslagen opname, met een kort bericht dat zegt wat veranderde.
Push / Pull	Je commits naar de gedeelde kopie sturen / die van anderen ophalen.
Branch (tak)	Een veilig zijspoor om iets te proberen zonder de hoofdversie te raken.
Clone	Je eigen lokale kopie maken van een gedeelde repo.

8.2 De dagelijkse lus

De dagelijkse Git-lus — vier stappen, herhaald



GitHub (of GitLab, of Azure DevOps in een Microsoft-omgeving) host de gedeelde kopie. Je kunt Claude in Code de stappen laten uitvoeren, maar begrijpen wat ze doen helpt.

Twee regels die de meeste Git-zorgen vermijden

- 1) Pull voor je begint, en commit vaak in kleine stukken — makkelijk te begrijpen en ongedaan te maken.
- 2) Nooit geheimen (API-sleutels, wachtwoorden) in een repo: eenmaal gepusht, beschouw ze als openbaar. Houd ze in een apart lokaal bestand dat Git negeert.

8.3 Werksessie — je eerste repo in 10 minuten

1. Open in Code een projectmap en zeg « stel deze map in als Git-repository ».
2. Maak een kleine wijziging, dan « commit met een duidelijk bericht ».
3. Maak een gratis GitHub-account; zeg « maak een private repo en push hem ».
4. Wijzig weer, commit, push — en zie de geschiedenis opbouwen.

9. In team werken aan hetzelfde project

Zodra twee mensen dezelfde bestanden raken, is er een manier nodig om hun werk te combineren zonder elkaar te overschrijven.

9.1 Documenten of code

- **Gedeelde documenten** (Word, Excel, slides): een SharePoint- of Drive-bibliotheek volgt de versies en toont wie wat veranderde. Vermijd hetzelfde bestand op dezelfde seconde te bewerken.
- **Gedeelde code** (scripts, apps, tekst): gebruik Git, gemaakt om de wijzigingen van meerdere mensen veilig samen te voegen.

9.2 De Git-lus in team

1. **Pull eerst** om op het werk van anderen voort te bouwen.
2. **Werk op een branch** genoemd naar de taak, niet op de hoofdversie.
3. **Open een pull request** — een collega kijkt ernaar, en dan voegt hij zich bij de hoofdversie.

Die review is geen bureaucratie — het is een tweede paar ogen, en daar leeft het gedeelde begrip. Wanneer twee mensen dezelfde regels wijzigen, vraagt Git welke versie te houden (een « merge conflict »); dat lijkt alarmerend maar is routine, en je kunt Claude vragen het uit te leggen en op te lossen.

10. Een zachte eerste stap in Code

Claude Code schrijft en draait echte software. Je hoeft de code niet te lezen of te schrijven — je beschrijft wat je wilt en Claude doet het.

10.1 Beschrijven, niet typen

« Voeg een knop toe die de tabel naar Excel exporteert » is een volledige instructie. Claude stelt de wijziging voor, jij keurt ze goed, het voert ze uit. Jij blijft de beslisser; Claude typt.

10.2 CLAUDE.md — het brein van het project

Elk Code-project zou een CLAUDE.md-bestand aan zijn wortel moeten hebben. Gelezen bij het begin van elke sessie, bevat het de structuur van het project, hoe het te draaien, je keuzes, je voorkeuren. Zeg na een goede eerste sessie « schrijf een CLAUDE.md die alles samenvat wat je leerde » en houd het bij.

10.3 De permissies — drie niveaus, niet twee

- **Elke keer vragen** (standaard): veilig, traag.
- **Bewerkingen automatisch accepteren** maar risicovolle commando's bevestigen: een goed compromis.
- **Alle permissies negeren** (snelle « vibe »-modus): alleen in een aparte map, met Git-back-up, vervangbaar — nooit je persoonlijke map.

10.4 Eerst plannen, dan bouwen

Voor alles wat niet triviaal is, vraag Claude te onderzoeken en een kort plan voor te stellen voor het één regel schrijft — « toon me twee opties en wacht op mijn keuze ». Een plan goedkeuren vermijdt lange omwegen en houdt je aan het roer.

10.5 Vibe coding — en zijn grenzen

Wat je hier doet heeft een naam: vibe coding — software bouwen door te beschrijven wat je wilt en op gevoel te sturen, in plaats van de code te lezen en te schrijven. Het schittert voor persoonlijke tools, prototypes en wegwerpscripts. Voor wat anderen nodig hebben, verstrengen de regels: houd het in Git, draai en test echt, en lever nooit wat je niet kunt verifiëren. Vibe-code vrij bij lage inzet; vertraag en controleer bij alles wat telt.

10.6 Claude Design — interfaces zonder code

Een neef van Code, in 2026 gelanceerd als research preview op claude.ai/design, bouwt Claude Design het uiterlijk van dingen — mock-ups, landingspagina's, eenvoudige websites en presentaties — uit een gewone beschrijving. Anders dan een beeldgenerator maakt het het echte ding: een werkende lay-out met knoppen waarop je echt kunt klikken, geen plaatje van een knop. Gebruik het wanneer je een pagina of een ontwerp wilt in plaats van software. Op Pro, Max, Team en Enterprise; als research preview zal het blijven veranderen.

11. Je eerste kleine agent

Een agent is gewoon Claude dat naar een doel toewerkt in meerdere stappen, vaak op een schema, en rapporteert. Geen groot systeem nodig — een nuttige, herhaalde taak is al een agent.

11.1 Wat een agent maakt

Drie ingrediënten: een duidelijk werk, een trigger (een uur of een gebeurtenis), en een manier om je het resultaat te zeggen. De rest is detail.

11.2 Een volledig geval — een ochtendbriefing

1. **Definieer het werk:** « Elke werkdag om 7 uur, lijst mijn vergaderingen van vandaag en de ongelezen mail gemarkeerd als belangrijk, en schrijf een samenvatting van 5 regels. »
2. **Maak er een opgeslagen taak van** zodat ze hetzelfde draait, met de connectors mail + agenda alleen-lezen.
3. **Plan ze** om 7 uur op werkdagen (je machine, of een altijd ingeschakelde machine, moet wakker zijn).
4. **Stuur ze waar je ze zult zien** — een e-mail naar jezelf, of een bericht op je telefoon.

Begin met een proefdraai

Draai ze een paar keer met de hand en lees de output kritisch voor je plant. Een agent die elke ochtend dezelfde kleine fout herhaalt is erger dan geen agent.

11.3 Laat hem langzaam groeien

Voeg één capaciteit tegelijk toe, en houd een kort notitiebestand bij over wat hij doet en hoe hem te repareren. Het is niet omdat Claude een functie in enkele seconden kan toevoegen dat ze zou moeten bestaan.

12. Tools en apps om te installeren

Je komt ver met alleen de desktop-app. Maar enkele gratis, standaardtools maken het bouwen vlotter. Installeer ze naargelang je ze nodig hebt.

Tool	Waarvoor het dient	Wanneer je het nodig hebt
Een editor (VS Code, of Notepad++)	Tekst- en codebestanden lezen en licht wijzigen.	Zodra je zelf de bestanden van een project opent.
Git + een GitHub-account	Versiegeschiedenis en teamwerk (h. 8–9).	Zodra een project telt of gedeeld wordt.
Python en Node.js	De motoren van de meeste kleine tools.	Wanneer Code iets bouwt om lokaal te draaien.
PuTTY / WinSCP	Verbinden met een server of Pi en er bestanden naartoe kopiëren.	Wanneer je taken draait op een ingeschakelde machine.
Een berichtenbot (Telegram)	Een agent je meldingen en rapporten laten sturen.	Wanneer je resultaten op je telefoon wilt.

Laat Claude ze installeren

Zeg « begeleid me om Git op Windows te installeren, stap voor stap, en controleer elke stap met mij ». Zo ook voor Python, Node, of de verbinding met een Pi.

13. Onderhoudbaar blijven

Het verschil tussen een leuk weekend en een betrouwbaar systeem over zes maanden zijn enkele discrete gewoonten.

- **Bewaar op twee plaatsen.** De source in Git, de belangrijke mappen gesynchroniseerd of gezippt. Eén kopie is geen back-up.
- **Commit en noteer al doende.** Kleine commits en een korte README: de toekomstige jij begrijpt een project snel.
- **Geheugen en sessienotities op grote schaal.** Een MEMORY.md en notities per project sturen elke nieuwe sessie.
- **Snoei skills en connectors** die je niet meer gebruikt — de dode zijn rommel en risico.
- **Weet te stoppen.** Voor je een functie toevoegt, vraag of ze haar plaats verdient. Aftrekken is een vak.

De test van één zin

Voor elk project zou je in één zin moeten kunnen zeggen wat het doet en waarom het bestaat. Zo niet, dan groeide het zonder richting — snoei voor je uitbreidt.

Woordenlijst van nieuwe termen

Term	In gewone woorden
Repository (repo)	Een projectmap plus heel zijn geschiedenis van opnamen, beheerd door Git.
Commit	Een opgeslagen opname, met een kort bericht dat de wijziging beschrijft.
Push / Pull	Je commits naar de online kopie sturen / die van anderen ophalen.
Branch (tak)	Een veilig zijspoor om iets te proberen zonder de hoofdversie te raken.
Merge conflict	Wanneer twee mensen dezelfde regels veranderden; Git vraagt wat te houden. Routine.
Pull request	Een verzoek om je branch samen te voegen, meestal eerst gereviewd.
GitHub / GitLab	Sites die de gedeelde kopie van een Git-repo hosten.
CLAUDE.md	Een bestand aan de wortel van een Code-project dat Claude zegt hoe het werkt.
MCP	De gemeenschappelijke standaard die connectors gebruiken om Claude met andere apps te verbinden.
GDPR	De EU-regels over persoonsgegevens — alles wat een levende persoon identificeert.
Agent	Claude dat naar een doel toewerkt in meerdere stappen, vaak gepland, en rapporteert.

Wat volgt

Als je Niveau Nul en dit Niveau Één-boek doorlopen hebt, weet je Claude goed in te stellen, echte resultaten te produceren, data verantwoord te beheren, je source veilig en gedeeld te houden, een stap in Code en Claude Design te zetten, en een kleine agent te draaien. Dat is het meeste van wat de meeste mensen nodig hebben.

De API — de stap voorbij dit boek

Je hebt misschien gehoord van de **Claude API** (onderdeel van het Claude Platform). De app, Cowork en Code laten je Claude allemaal gebruiken via een venster dat anderen bouwden. De API laat een eigen programma rechtstreeks met Claude praten — afgerekend per token — om je eigen apps en agents te bouwen. Het is technischer, vandaar zijn plaats voorbij dit boek; maar het is dichterbij dan het lijkt — zodra een kleine tool die je in Code maakt Claude zelf aanroept, gebruikt hij al de API.

En houd de twee gewoonten van Niveau Nul dichtbij: **laat Claude eerst vragen, en vertrouw nooit een zelfverzekerd antwoord zonder het te verifiëren**. Ze tellen méér, niet minder, naarmate wat je bouwt groeit.

Door Hans Tandt